

Manuel utilisateur de autolisp-doc

OpenAI

April 27, 2026

Contents

1	Objet	1
2	Structure du sous-projet	2
3	Chargement	2
4	API publique	2
4.1	Documentation	3
4.2	Description	3
4.3	Recherche par nom	4
4.4	Point d'entrée <code>help</code>	4
4.5	Analyse de formes	5
5	Recherche textuelle	6
6	Génération de la base documentaire	7
7	Tests	8
8	Limitations actuelles	8

1 Objet

`autolisp-doc` fournit une petite couche de documentation interactive pour AutoLISP, alimentée par une base locale de référence extraite de la documentation Autodesk.

Le module couvre pour l'instant :

- la consultation structurée de la doc d'une fonction ;

- la description d'objets Lisp ;
- la recherche par nom de fonction ;
- la recherche textuelle dans la base documentaire ;
- une analyse simple de formes Lisp ;
- un point d'entrée pratique `help`.

2 Structure du sous-projet

`src/autolisp-ref.lsp` base documentaire générée ;

`src/autolisp-doc.lsp` API utilisateur ;

`tests/` tests unitaires ;

`docs/` documentation utilisateur ;

`scripts/build_autolisp_ref.py` générateur de la base.

3 Chargement

Depuis la racine du dépôt :

```
(setq *outils-autolisp-path* "/chemin/vers/outils-autolisp")
(setq *verbose* T)
(load (strcat *outils-autolisp-path* "/loader.lsp"))
```

Pour charger seulement `autolisp-doc` :

```
(setq *autolisp-doc-path* "/chemin/vers/outils-autolisp/autolisp-doc")
(setq *verbose* T)
(load (strcat *autolisp-doc-path* "/loader.lsp"))
```

4 API publique

Les fonctions principales existent maintenant sous deux familles de noms :

publiques `documentation`, `describe`, `apropos`, `apropos-list` ;

préfixés ad- `ad-documentation`, `ad-describe`, `ad-apropos`, `ad-apropos-list`.

4.1 Documentation

4.1.1 (documentation symbol doc-type)

Retourne une chaîne formatée décrivant une fonction documentée.

Types de documentation actuellement pris en charge :

- T
- function
- FUNCTION

Exemple :

```
(documentation 'open 'function)
```

Le résultat inclut notamment :

- le titre ;
- le résumé ;
- la signature ;
- les arguments ;
- les valeurs de retour ;
- l'historique ;
- les exemples ;
- l'URL de la source Autodesk.

4.2 Description

4.2.1 (describe object)

Affiche une description lisible de l'objet et retourne l'objet lui-même.

Comportement principal :

symbole documenté affiche la documentation formatée ;

symbole lié mais non documenté affiche sa valeur et son type ;

liste affiche sa longueur et sa forme imprimée ;

autre objet affiche le type et la forme imprimée.

Exemples :

```
(describe 'open)
(describe '(1 2 3))
(describe 42)
```

4.3 Recherche par nom

4.3.1 (apropos pattern)

Affiche une ligne de résumé par symbole documenté correspondant au motif.

4.3.2 (apropos-list pattern)

Retourne la liste des symboles documentés correspondant au motif, sans les afficher.

Le motif utilise une syntaxe de wildcard simple compatible `wcmatch` :

`*` n'importe quelle suite de caractères ;

`?` un caractère ;

sans wildcard explicite, la recherche est traitée comme un `contains`.

Exemples :

```
(apropos "open")
(apropos "str*")
(apropos-list "*hash*")
```

4.4 Point d'entrée help

4.4.1 (help)

Affiche l'index des fonctions d'aide disponibles.

4.4.2 (help 'symbol)

Fait une recherche par nom, comme `apropos`.

4.4.3 (help "text")

Fait une recherche textuelle dans la base de documentation complète.

Exemples :

```
(help)
(help 'open)
(help "unicode")
```

Une variante préfixée est aussi disponible :

```
(ad-help "unicode")
```

4.5 Analyse de formes

4.5.1 (ad-categorize form)

Analyse une forme Lisp, en particulier une forme `defun`, et retourne la liste des familles de symboles documentés qu'elle appelle.

Exemples de catégories :

- AUTOLISP
- AUTOLISP/DCL
- AUTOLISP/ACTIVE-X
- AUTOLISP/VISUAL-LISP-IDE

Exemple :

```
(ad-categorize
 '(defun demo ()
   (action_tile "accept" "(done_dialog 1)")
   (vl-load-com)
   (alert "ok")))
; => (AUTOLISP/DCL AUTOLISP/ACTIVE-X AUTOLISP)
```

Alias :

- (categorize form)

4.5.2 (ad-free-variables form)

Retourne les variables libres d'une forme, c'est-à-dire les symboles utilisés comme valeurs mais non liés localement ni déclarés comme paramètres.

La vl sait notamment reconnaître :

- `defun` ;
- `lambda` ;
- `foreach` ;
- `setq` ;
- `quote` / `function` pour éviter de descendre dans des données ordinaires.

Exemple :

```
(ad-free-variables
 '(defun demo (a / b)
   (setq b (+ a x))
   (foreach item items
     (setq z (+ item y b)))
   (list a b z x y items)))
; => (X ITEMS Z Y)
```

Alias :

- `(free-variables form)`

5 Recherche textuelle

La recherche textuelle inspecte plusieurs champs de chaque entrée :

- `nom` ;
- `titre` ;
- `résumé` ;
- `signature` ;
- `arguments` ;

- retour ;
- historique ;
- exemples ;
- URL source.

Cela permet par exemple de retrouver les fonctions ayant une note sur Unicode, sur Windows/Mac, ou sur un argument particulier.

6 Génération de la base documentaire

Le fichier `src/autolisp-ref.lsp` est généré à partir de la documentation Autodesk par :

```
python3 scripts/build_autolisp_ref.py
```

La base actuelle stocke, pour chaque fonction :

- un nom canonique ;
- un résumé ;
- une famille source (AutoLISP, AutoLISP/DCL, AutoLISP/ActiveX, AutoLISP/Visual LISP IDE) ;
- une signature ;
- les arguments ;
- les valeurs de retour ;
- l'historique ;
- les exemples ;
- l'URL d'origine.

7 Tests

Depuis `autolisp-doc/` :

```
make test-bricscad
```

Les tests valident notamment :

- la présence d’entrées documentaires ;
- le contenu de `documentation` ;
- la recherche avec `apropos-list` ;
- les résumés affichés par les fonctions d’aide via leurs helpers internes ;
- la recherche textuelle de `help` ;
- la catégorisation de formes ;
- l’extraction de variables libres ;
- la cohérence des alias `ad-`.

8 Limitations actuelles

- la syntaxe de recherche n’est pas une expression régulière complète ;
- `documentation` ne couvre pas encore d’autres types de documentation que les fonctions ;
- il n’y a pas encore d’`inspect` interactif ;
- la base vient actuellement de la source Autodesk ENU accessible en HTTP brut, même si les GUID correspondent à la référence affichée côté FRA.